1. General Information

Auth Service — микросервис, отвечающий за регистрацию и аутентификацию пользователей, проверку учётных данных и управление пользовательскими сессиями.

Technical Specifications

Parameter	**Value**
Protocol	HTTP/HTTPS
External Port	443 (HTTPS)
Internal Port	8080 (HTTP, behind NGINX)
Database	PostgreSQL
Cache / Session Storage	Redis
Authentication	JWT
Logging	Go log/slog (JSON Handler)
Data Format	JSON

TLS termination is handled by NGINX. Internal communication between NGINX and Auth Service uses HTTP.

2. API Endpoints

2.1. POST /api/reg

Purpose: Register a new user.

Request:

Content-Type: application/json

{

"name": "Pavel",

"email": "user@example.com",

"password": "secure_password"

}

Request Processing:

1. Auth Service receives JSON from Frontend.
2. Validates the input data.
3. Checks whether a user with the specified email already exists in PostgreSQL.

SQL Query — User Lookup:

SELECT id

FROM users

WHERE email = $1;

4. If the user already exists, returns 409 Conflict.

Response — 409 Conflict:

```
{
  "error": "USER_ALREADY_EXISTS",
  "message": "User with this email already exists"
}
```

5. If the user does not exist, Auth Service hashes the password.
6. Creates a new user record in PostgreSQL.

SQL Query — User Creation:

```
INSERT INTO users (name, email, password_hash)
VALUES ($1, $2, $3)
RETURNING id, name, email;
```

7. Generates a JWT after successful registration.
8. Creates a user session in Redis with the corresponding TTL.
9. Returns the registration result to Frontend.

Redis Structure:

Key:

auth:session:<user_id>:<session_id>

Value:

<JWT_TOKEN>

TTL: session expiration time.

Response — 201 Created:

```
{
    "status": "success",
    "message": "User registered successfully",
    "user": {
        "id": 1,
        "name": "Pavel",
        "email": "user@example.com"
    },
    "access_token": "<JWT_TOKEN>",
    "token_type": "Bearer",
    "expires_in": 900
}
```

2.2. POST /api/auth

Purpose: Authenticate an existing user.

Request:

Content-Type: application/json

{

"email": "user@example.com",

"password": "secure_password"

}

Request Processing:

1. Auth Service receives JSON from Frontend.
2. Validates the input data.
3. Searches for the user in PostgreSQL.

SQL Query — User Lookup:

SELECT id, name, email, password_hash

FROM users

WHERE email = $1;

4. If the user is not found, returns 401 Unauthorized.
5. If the user is found, compares the provided password with the stored password hash.
6. If the passwords do not match, returns 401 Unauthorized.
7. If authentication is successful, generates a JWT.
8. Creates a new user session in Redis.
9. Returns the JWT to Frontend.

Redis Structure:

Key:

auth:session:<user_id>:<session_id>

Value:

<JWT_TOKEN>

TTL: session expiration time.

Response — 200 OK:

```
{
    "status": "success",
    "message": "Authentication successful",
    "user": {
        "id": 1,
        "name": "Pavel",
        "email": "user@example.com"
    },
    "access_token": "<JWT_TOKEN>",
    "token_type": "Bearer",
    "expires_in": 900
```

}

3. HTTP Status Codes

Code	Description
200 OK	Successful authentication
201 Created	Successful registration
400 Bad Request	Invalid JSON or input data
401 Unauthorized	Invalid credentials
409 Conflict	User already exists
500 Internal Server Error	Internal service error
503 Service Unavailable	Database or Redis unavailable

4. JWT Authentication

JWT is used to identify users during subsequent requests to Backend.

Example JWT Payload:

```
{
    "sub": "1",
    "session_id": "session_uuid",
    "iat": 1793900000,
```

"exp": 1793900900

}

- sub — user ID.
- session_id — user session identifier.
- iat — token creation timestamp.
- exp — token expiration timestamp.

Recommended signing algorithms: HS256 or RS256.

Default Access Token lifetime: 15 minutes.

For authenticated requests, Frontend sends:

Authorization: Bearer <JWT_TOKEN>

5. Logging

Logging is implemented using Go standard library log/slog with JSONHandler.

Example:

{

"time": "2026-11-05T12:00:00Z",

"level": "INFO",

"msg": "User authenticated successfully",

"user_id": 1,

```
    "service": "auth-service"
}
```

Events to log:

- Incoming HTTP requests.
- Successful registration.
- Successful authentication.
- Failed authentication attempts.
- PostgreSQL errors.
- Redis errors.
- JWT generation errors.

Security requirements:

Passwords, password hashes and JWT tokens must never be written to logs.

6. Additional Endpoints (Future Implementation)

Method	Endpoint	Description
POST	/api/logout	Terminate the current user session
POST	/api/refresh	Refresh an expired Access Token